Webalkalmazások elemei

Témakörök

- Az internet kialakulása
- Webalkalmazások elemei
- HTML története
- Kétrétegű kliens-szerver architektúra
- HTTP protokoll kérés - válasz
- Web-technológiák

Az internet kialakulása

„Az **internet** az egész világot *behálózó számítógép-hálózat*.

Az internet elődjét az ARPANET-et (Advanced Research Projekt Agency, Amerikai Védelmi Minisztérium Kutatóintézete) az 1960-as években fejlesztették ki.

Mivel katonai célból fejlesztették ki, ezért fontos volt, hogy ha egy esetleges *támadás esetén egyes részei megsemmisülnek*, akkor a megmaradó rész továbbra is üzemképes legyen (atomcsapás, terrorista merénylet), tovább használható.

Az internet kialakulása

Ennek megoldását a következő eszközökkel érték el:

Az *adatátvitel* módját *hardverfüggetlen* módon *szabványosították*: ez azt jelenti, hogy különböző eszközök köthetők a hálózatba („internetes hűtőszekrényektől a mobiltelefonig” stb.).

Működése esetén az adatokat *csomagokra bontják* és ezeket ellátják *azonosító adatokkal* és egyenként küldik a hálózatón: *feladó*, *címzett, csomag sorszáma*.

Az internet kialakulása

A *csomagok* irányítását *speciális számítógépekkel* (router – útvonal-választók) végzik.

A *beérkező csomagokat* a célállomáson futó programok *összerakják*, sérülés esetén ismétlést kérnek a küldő számítógéptől.

A 70-es évekig több egymástól elszigetelt *csomagkapcsolt hálózat* működött, később egységesítsék az *adatátviteli módszereket.*

Az internet kialakulása

Kialakult a ma is használt *Internet Protocol* (IP).

A **protokoll** a *hálózati adatátvitel* módját *leíró szabályok összessége.*

A 80-as években csatlakoztak a hálózathoz az *egyetemek, főiskolák, kutatóintézetek,* valamint az *állami hivatalok*.

Napjainkban az internet hálózat mérete már exponenciális ütemben nő….

Az internet kialakulása

Az internet mai arculatát meghatározó *World Wide Web* (világháló) *Tim Berners-Lee* (szül.: London, 1955) elképzelésén alapul.

1989-ben a genfi CERN (Nukleáris Kutatások Európai Szervezet) kutatójaként egy olyan *hipertext alapú információs rendszer* létrehozására tett javaslatot, amely *egységes grafikai felületen* lenne képes kezelni a *különböző helyeken* levő *számítógépeken szétszórt információkat*.

Az internet kialakulása

Központi gondolata volt:

- *hipertext*, szöveges információt hordozó dokumentumtípus létrehozása (A „hiper” dokumentumok a szövegbe ágyazott, más hasonló fájlokra mutató hivatkozásokat, ún. *linkeket* is tartalmaznak).
- hipertext-dokumentumok az *internet kiszolgáló gépein elosztottan* helyezkednek el,

Az internet kialakulása

- szövegben található *hivatkozások* révén *egymáshoz kapcsolódnak, és világméretű dokumentumszövedéket* (web), hatalmas összekapcsolt szöveget, *hipertextet alkotnak*.

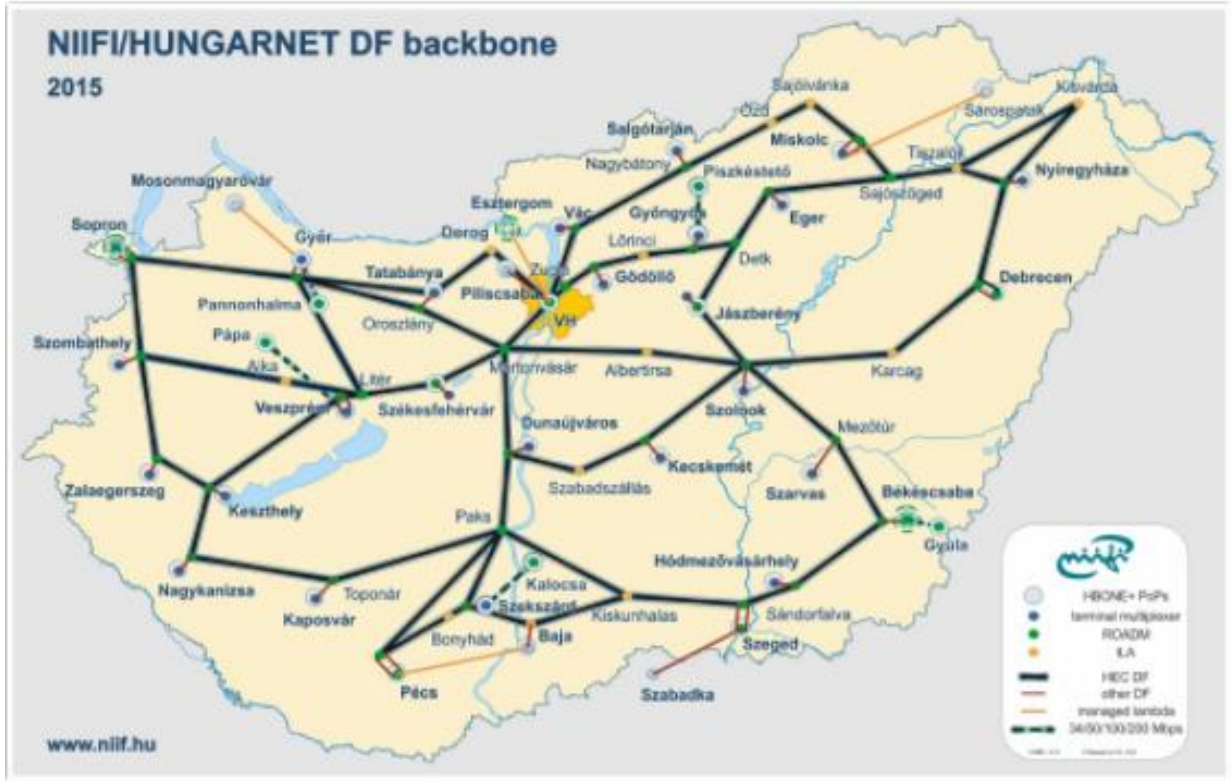
Az internet kialakulása

A Nemzeti Információs Infrastruktúra Fejlesztési Program (NIIF) a magyarországi kutatói hálózat fejlesztésének és működtetésének állami programja.

Az NIIF gerinchálózata a **HBONE**, amely a *hazai akadémiai közösség számítógéphálózata.*

Feladata: ez szolgálja ki a *hazai felsőoktatást*, *kutatásfejlesztést*, *könyvtárfejlesztést* és *közgyűjteményeket*, valamint számos egyéb *közintézményt is*.

Az internet kialakulása



Forrás:

NIIF / Hungarnet DF backbone

2015

URL: https://digitalisjoletprogram.hu/hu/tartalom/niif-nemzeti-informacios-infrastruktura-fejlesztesi-program

Webalkalmazások elemei

World Wide Web *négy alappilléren nyugszik*:

- *HTML dokumentumjelölő* nyelv,
- *Kliens-szerver architektúra*, amely *weblapok elosztott tárolását*, és interneten keresztül *történő letöltését*, *megjelenítését biztosítja*,
- *HTTP protokoll*, amely *webszerver és kliens* közötti *adatcsere szabványosságáért* felelős,

Webalkalmazások elemei

- *URL címzési rendszer*, amely a weblapok *címezhetőségét, letölthetőségét,* és a közöttük lévő *kapcsolatok leírását* biztosítja.

HTML - HyperText Markup Language

- „A HTML a Web elsődleges leíró nyelve.”
- „[...] egy szemantikai szintű leíró nyelv és a kapcsolódó szemantikai szintű alkalmazásprogramozási interfészek a Weben elérhető oldalak készítéséhez, melyek a statikus dokumentumoktól a dinamikus alkalmazásokig terjednek.”

Forrás: HTML Living Standard (utolsó módosítás: 2021. január 7.)
https://html.spec.whatwg.org/

WHATWG = Web Hypertext Application Technology Working Group

HTML története

HTML (HyperText Markup Language = hipertext jelölő nyelv)

- A HTML az SGML szintaxisára épülő *jelölőnyelv*,
- Relációs jelek közé zárt (<>) HTML-jelölők az elvégzendő műveleteket írják le,
- HTML 1.0 verziója csak *szöveges tartalmak* leírását tette lehetővé,

HTML története

- 1995-ben megjelent *HTML 2.0*, amely számos *szövegformázási lehetőséget*, és többek között *képek csatolását* is biztosította.
- jelölőnyelv igazi jelentősége az *űrlapok* készítésére alkalmas, `<FORM>` elem megjelenésében rejlett,
- A `HTML 2.0` biztosította először, hogy a *weblapok háttérben futó alkalmazások* kezelőfelületeiként működjenek,

HTML története

- 1997: HTML 3.2 (lehetővé vált *a táblázatok*, *matematikai képletek* és v*ezérlőelemek* használata),
- 1999: HTML 4.0 (stíluslapok, frame-ek használata, továbbfejlesztett *szöveg* és *táblázatirány meghatározás*, *űrlapok*, stb.)
- 2000: XHTML 1.0 (Extensible Hypertext Markup Language, Bővíthető Hiperszöveges Leírónyelv),

HTML története - XHTML

Az *XML alkalmazásként* definiált HTML szigorúbb szabályokat ír elő a dokumentumok számára, így azok feldolgozása egyszerűbb.

- Különösen lényeges ez a hagyományos asztali gépekhez képest korlátozott lehetőségekkel bíró eszközökénél (pl.: mobil eszközöknél).
- Az XHTML, lehetővé teszi az XHTML kombinálását más XML alkalmazásokkal.

HTML története - XHTML

XHTML előnyei:

- Könnyen áttekinthető.
- Egyszerűen szerkeszthető.
- Könnyen érvényesíthető *(validálható)* standard XML eszközökkel.

Az XHTML oldalak a HTML 4.0 szabványt támogató szerkesztőprogramokkal is szerkeszthetők.

HTML története - XHTML

- 2000-2004: különböző technológiák jelentek meg: Flash, AJAX és kialakult az XHTML 2.0.
- 2004: létrejött a WHATWG csoport = Web Hypertext Application Technology Working Group

  Nem tetszett nekik az XHTML 2.0, inkább a HTML-t szabvány megújításával foglalkoztak.

HTML története - HTML

- 2007: A WHATWG munkáját a W3C **HTML 5** néven a (World Wide Web Consortium 1994 alakult, Web lehetőségeit bővítő, javító, protokollok, rendszerek kidolgozásával foglalkozó szervezett) adoptálta.
  - Megmaradt a visszafelé kompatibilitás.
  - Új elemeket vezettek be, melyek egyszerűbbé tették a *web- és mobilalkalmazások* fejlesztését.

HTML története - HTML

- 2009: A W3C nem újította meg az XHTML 2.0-t (HTML 5 vitték tovább)
- 2010: A Google és Apple hatására megnőtt a *HTML 5* iránti kereslet és divatos fogalommá vált a fejlesztők körében a HTML 5.

Miért van rá szükség?

- Visszafelé kompatibilis (a HTML 3, 4-ben megírt weboldalak működnek)

HTML története – HTML5

- A legtöbb böngésző ugyanúgy jeleníti meg a weblapokat.
- Beépített hibakeresési algoritmusok (hiba esetén a böngészők fel tudják dolgozni).
- Új elemek és tulajdonságok.
- Beépített API-k segítik a webes alkalmazások fejlesztését.
- Audió és videó, grafika támogatás (egyszerűen lehet hang és videót anyagokat beilleszteni).
- A HTML 5 sok, korábban külön beépítendő plugin-t kivált.

HTML története – HTML5

Mely böngészők támogatják a HTML5?

- Chrome (52 +)
- Opera (37+)
- Firefox (48+)
- Safari (10.0+)
- Microsoft Edge (14+)

A HTML5 teszt pontszáma azt jelzi, hogy böngészője mennyire támogatja a HTML5 szabványt és a kapcsolódó specifikációkat (555-ből 528-t.)
http://html5test.com/

Webalkalmazások elemei

Webszerver, webkliens

A *WWW szolgáltatásnak* az internet gépein elosztottan tárolódó, de bárhonnan elérhető, és bármely hoston megjeleníthető szöveges dokumentumok kezelését kellett biztosítania.

Ezt a feladatot *Tim Berners Lee* kétszintű **kliens szerver architektúra** implementálásával oldotta meg.

Webalkalmazások elemei

A *webszerver* feladata: a *hipertext oldalak tárolása* és a *webkliensek* kiszolgálására szolgáló rendszer.

A kiszolgáló folyamatosan „figyeli” a kliensek felől érkező, *kéréseket*, amelyeket a kért weblapot, vagy valamilyen egyéb adatot tartalmazó *válasszal szolgál ki.*

A (web**)kliensek** (*böngészők)* biztosítják a *szolgáltatás és a felhasználók közötti kapcsolatot.*

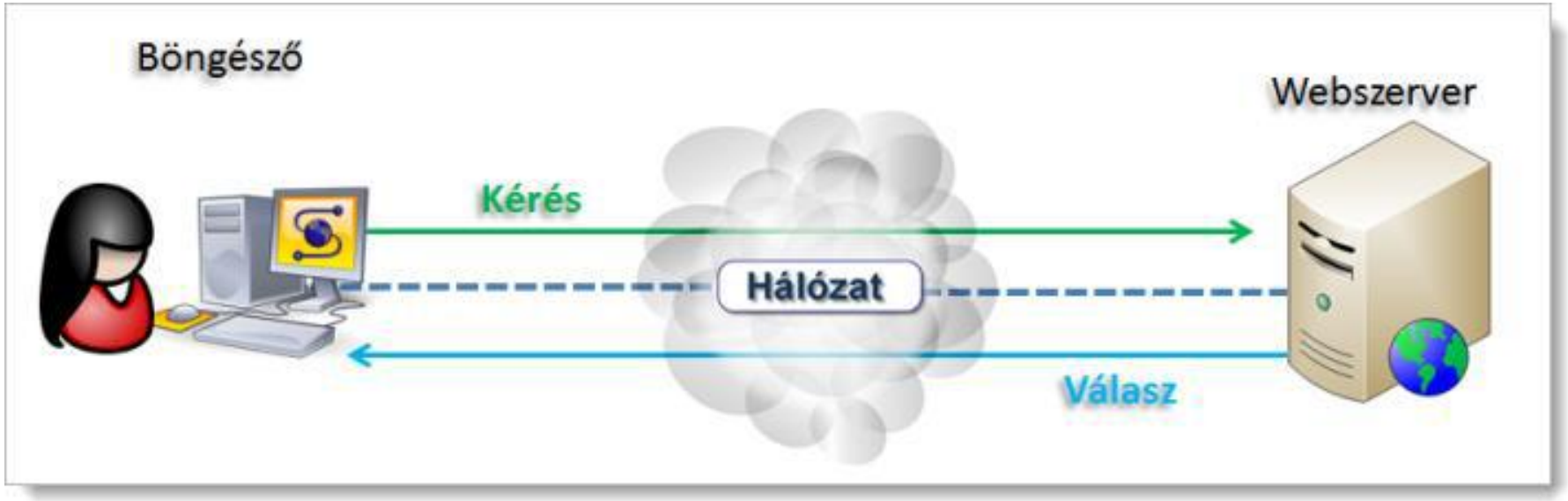
Webalkalmazások elemei

- A felhasználó gépén *futó böngésző,* biztosítja a *szolgáltatás kezelőfelületét.*
- Ha a felhasználó valamilyen *weblap letöltését kéri*, akkor a *böngésző* felveszi a kapcsolatot a *szerverrel*, és elküldi a weblapra vonatkozó *kérést*.
- A kiszolgáló által *visszaküldött oldalt*, vagy *üzenetet* a felhasználó számára értelmezhető formában láthatóvá teszi képernyőn.

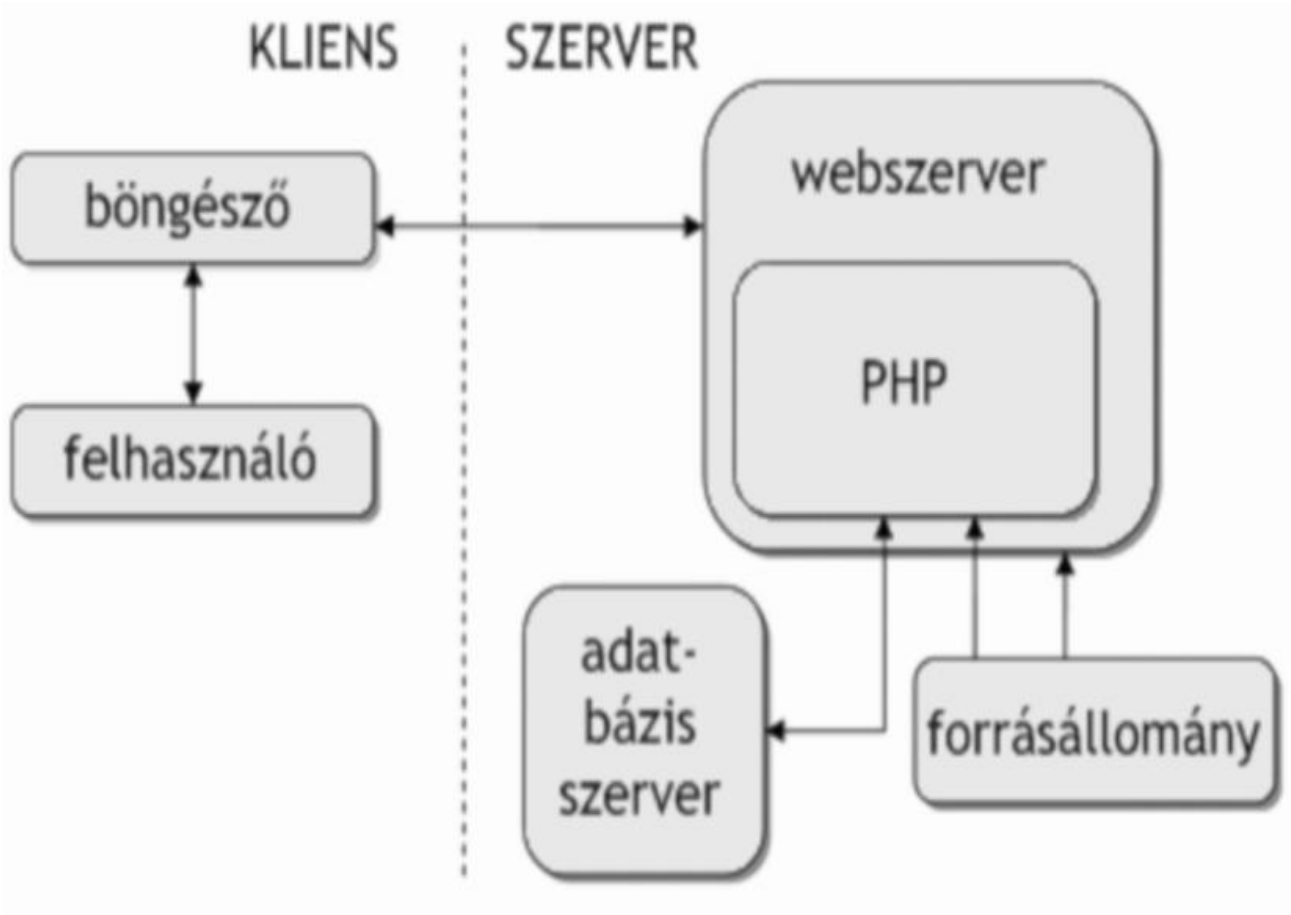
Webalkalmazások elemei

- A weblap megjelenítésekor értelmezi a szövegbe ágyazott HTML-jelölőket, és azoknak megfelelő formátumban jeleníti meg a dokumentumot.

Kétrétegű kliens szerver architektúra



Webalkalmazások elemei



Webalkalmazások elemei

Webhely

HTML lapok és más *médiafájlok gyűjteménye*, amelyek egy *webszerveren helyezkednek* el és *láthatók* az *ügyfelek* számára.

URI (Uniform Resource Idenfitier)

Az *URI* (Uniform Resource Idenfitier, egységes erőforrás-azonosító), egy olyan karaktersorozat, amely *azonosít* egy *erőforrást*.

Webalkalmazások elemei

Az URI az *erőforrást* kétféleképpen azonosíthatja:

- *hely* (URL Uniform Resource Locator) vagy
- *név* (URN Uniform Resource Name) szerint.

Az URL-t *webcímnek* is nevezzük, és a következő részekből tevődik össze.

Webalkalmazások elemei

URL-címzés

Az URL (Uniform Resource Locator) olyan *egyedi azonosító rendszer*, mely a világ különböző web szerverein tárolt erőforrások *egyedi módon, egyértelműen azonosíthatók*.

Az URL-t *webcímnek* is nevezzük.

Az URL címek általános leírása:

[*protokoll*://]*host*[[/*útvonal*/][*fájl*][**?***paraméterek*]]

Webalkalmazások elemei

- A *protokoll* jelöli a *kliens* és *böngésző* közötti *üzenetváltás szabályrendszerét.*
- A *host* a webszerver számítógép *IP-címe*, vagy *domain* neve, a dokumentumot tároló *host egyértelmű azonosítója*.
- Az *útvonal* határozza meg, hogy a szerver melyik *mappájában* található a kért állomány.
- *Fájl*: ez az opció jelzi, hogy *melyik fájljára* van szükség.

[*protokoll*://]*host*[[/*útvonal*/][*fájl*][?*paraméterek*]]

HTTP protokoll ill. HTTPs

HyperText Transfer Protocol = hipertext átviteli protokoll.

- *HTTPS* (Hypertext Transfer Protocol Secure) - a HTTP protokoll biztonságos verziója, amely a *SSL /TLS protokoll* titkosításhoz és hitelesítéshez.

A HTTPS hozzáteszi *titkosítás, hitelesítés* és *sértetlenség* a HTTP protokollhoz:

- *Titkosítás: SSL /TLS protokoll* megakadályozza az interneten küldött adatok harmadik fél általi elfogását és olvasását.

HTTP protokoll - HTTPs

- *Hitelesítés:* Webhely *SSL /TLS igazolás* tartalmaz egy *nyilvános kulcs*, hogy egy webböngésző megerősítheti, hogy a szerver által elküldött dokumentumokat valaki *digitális aláírással* írta alá, aki rendelkezik a megfelelő *magánkulcs*.
- *Integritás:* A kiszolgáló kiszámítja a *kriptográfiai kivonat* alapján, hogy a dokumentum *integritása sértetlen.*

HTTP protokoll ill. HTTPs

- A *titkosítás*, a *hitelesítés* és az *integritás* ezen garanciái együttesen teszik a HTTPS-t sok *biztonságosabb protokoll* az interneten való böngészéshez és üzletvitelhez, mint a HTTP.”

További információ: Mi a HTTPS?

URL: https://www.ssl.com/hu/gyik/mi-a-https/

HTTP protokoll

HTTP

- *szöveges, kérés-válasz alapú,*
- *általános, állapotmentes protokoll,*

amely a *kliens- és szerver* közötti *adatforgalom módját*, az ***üzenetek*** *lehetséges* tartalmát határozza meg.

A *kliens* és a *szerver* ún. **üzenetek** formájában kommunikálnak egymással.

HTTP protokoll

Üzenetek: *fejrészre* és *üzenettestre* bonthatók.

- *Fejrész*: a utasításokat és metaadatokat tartalmaz.
- *Üzenettest:* különböző *adatokat* (pl.: felhasználói nevet és a jelszót, vagy a szerver által visszaküldött fájlt) tartalmaz.

HTTP protokoll - kérés

HTTP kérés formátuma

A *kapcsolat felépítése* mindig a *kliens kérésével indul.*

Az *üzenet* első sora az ún. *kéréssor,* amelyet több, ún. *header* sor követhet.

Minden *header* sor egy *fejlécmező*-t tartalmaz.

Az üzenet *fejlécét,* és az azután következő **üzenettestet**, üres sor választja el.

HTTP protokoll - kérés

A *kérés* általános formátuma az alábbi:

1 *kéréssor***<CRLF>**

[*header_sor***<CRLF>…]**

<CRLF>

[*üzenettest*]

Legegyszerűbb esetben a *kérés* egyetlen *kéréssorból* áll.

A **kéréssor** tudatja a szerverrel, hogy mit kér tőle a kliens.

(CRLT kocsi vissza-soremelés)

HTTP protokoll - kérés

Első eleme egy **metódus**, majd egy **URL-cím,** majd a HTTP protokoll **verziószáma** követ. A sor formátuma a következő:

metódus URL-cím HTTP/*verzió*

- Az **URL-cím** *GET* esetén a *kért fájl URL-címe*, *POST esetén* az az állomány, ami f*eldolgozza a küldött adatokat.*
- A *verzió,* pedig a HTTP protokoll kliens által használt verziójának száma (0.9, 1.0, 1.1).

HTTP protokoll – kérés - kitérő

HTTP protokoll *nyolcféle metódust* definiál, amely az *erőforráson végzendő műveletet* határozzák meg.

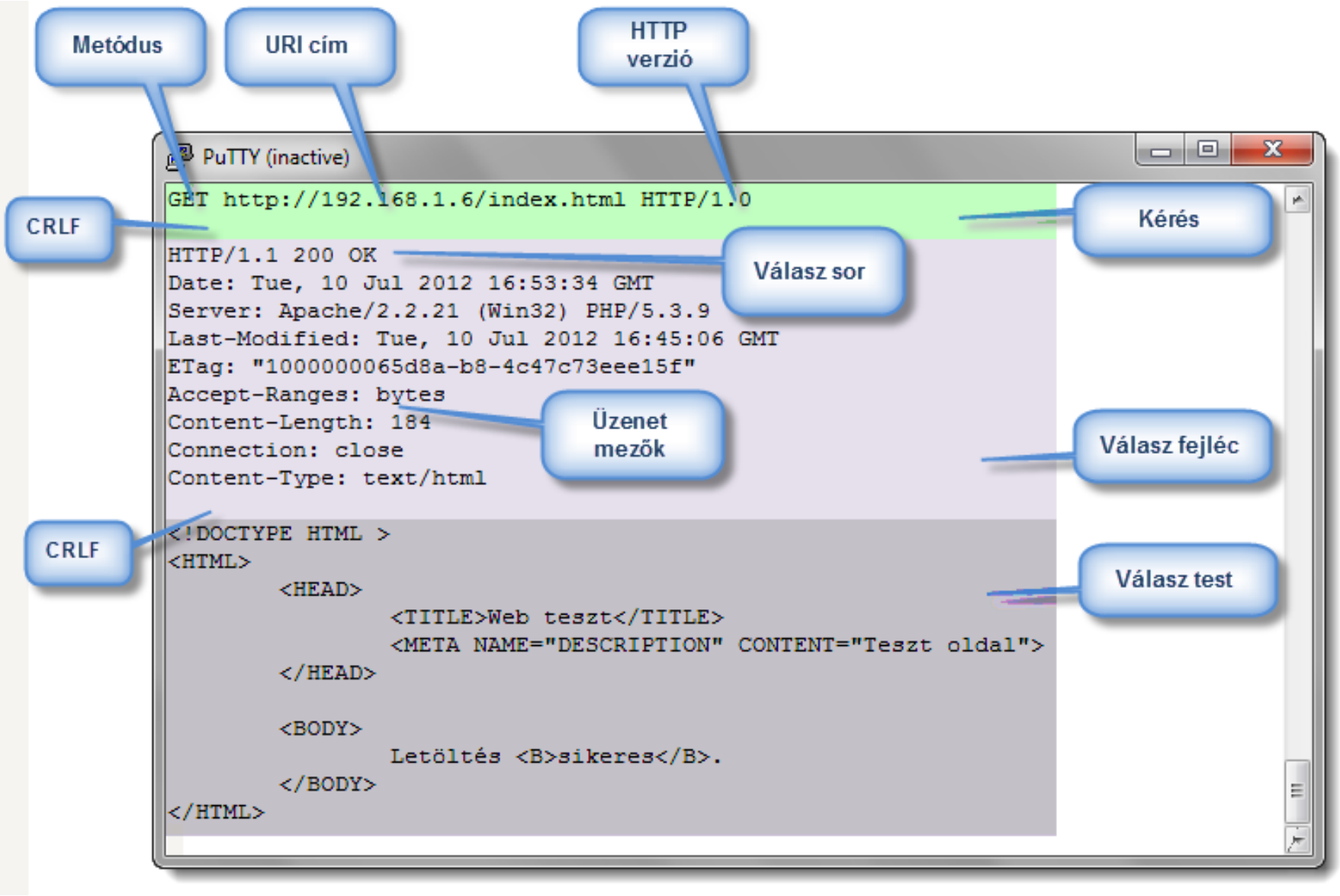
- **GET** - megadott *erőforrás letöltését* kezdeményezi.
- **HEAD** - Ugyanazt adja vissza, mint a GET, csak magát az *üzenettestet hagyja ki a válaszból*.
- **POST -** a feldolgozandó adatot küld a szerverre. Pl.: HTML űrlap tartalmát. Az adatot az üzenettest tartalmazza.

HTTP protokoll – kérés - kitérő

- **PUT** - Feltölti a megadott erőforrást.
- **DELETE** - Törli a megadott erőforrást.
- **TRACE** - Visszaküldi a kapott kérést.
- **OPTIONS** - Visszaadja a szerver által támogatott HTTP metódusok listáját.
- **CONNECT** - Átalakítja a kérést transzparens TCP/IP tunnellé.

HTTP p

HTTP kérés



GET http://192.168.1.6 /index.html HTTP/1.0

HTTP protokoll - válasz

HTTP válasz

A szerver válasz: *fejlécből* és *üzenettestből* áll.

A fejléc a **válaszsorral** kezdődik, amit *kocsi vissza-soremelés* (CRLF) karakter zár.

A **válaszsor** a szerver által használt *protokollverzióval* kezdődik (HTTP/1.1), majd a *válaszkód* (200), (amit a kérés teljesítésének sikerét jelző) és az ahhoz tartozó *üzenet* (OK) követ.

HTTP protokoll - válasz

A *válaszsor után* a következik **fejléc** mezők.

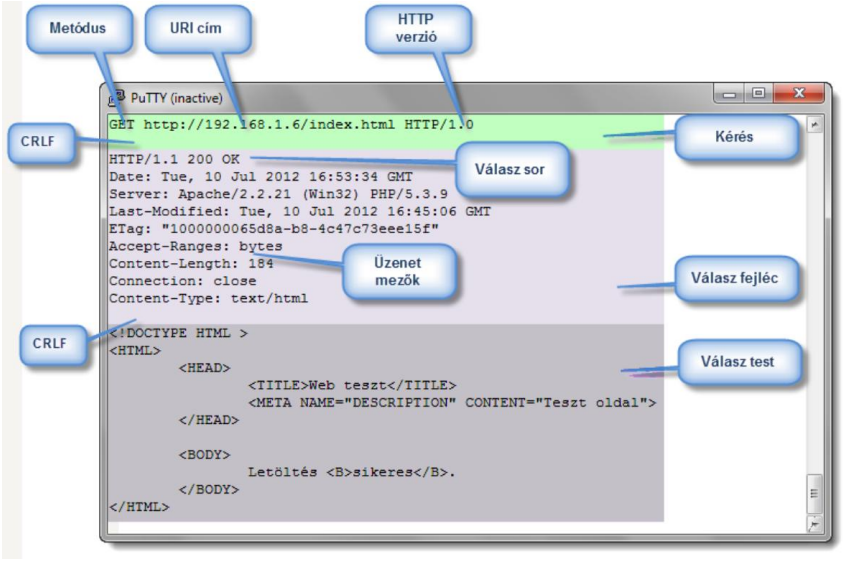
Ezek a *szerverről* és a visszaküldött *adatokról* tartalmaznak *információkat,* és mindig *név*:*érték* formátumú.

Server: Apache/2.2.21 (Win32) PHP/5.3.9 fejlécmezője pl.: a webszerver adatait:

- a **Date** a kérés teljesítésének időpontját,
- a **Last-Modified** a kért fájl *utolsó módosításának időpontját*,
- az **ETag** a fájl ellenőrzőösszegét,

HTTP protokoll - válasz

- a **Content-Length,** a *válasz hosszát*,
- az **Accept-Ranges,** az előző *érték mértékegységét*,
- a **Content-Type,** pedig az *adatok típusát* adja meg.



Hypertext Transfer Protocol (HTTP)

- A HTTP *aszinkron kérés-válasz alkalmazás-réteg protokoll*.
- Az ügyfél *kérési üzenetet* küld a szervernek.
- A kiszolgáló ezután *válaszüzenetet küld* az ügyfélnek.
- HTTP egy *pull* protokoll, az ügyfél egy oldalt húz le a szerverről.
- Az üzenet szintaxisát a *HTTP specifikáció* határozza meg.

Webalkalmazások

Mit tartalmaz egy *Webhely*?

Webhely: statikus állományok gyűjteménye, amely *HTML files*, *kép* és más *formátumú állományokat* tartalmaz.

És a Webalkalmazás?

Webalkalmazás: webhely + *szerveroldali programok*

Mi a különbség a *webhely* és a *webalkalmazás* között?

- *webhely: csak statikus tartalmat* szolgáltat,

Webalkalmazások

- *webalkalmazás: dinamikus tartalom* előállítására is képes.

Mi a feladata a *webalkalmazás -n*ak:

- dinamikus tartalom előállítását *szerveroldalon futó programok* végzik,
- különböző *programozási nyelvekben* lehet elkészíteni (pl.: Perlben megírt szkriptek, C/C++ stb.)

A webszerver szerveroldali programot a külön *folyamatként* vagy *folyamatszálként* lefuttatja minden *egyes kérésre*, a program *eredményét* visszaküldi a böngészőnek.

Webalkalmazások

A *szerveroldali program* általában *adatokat fogad*, és azok feldolgozása után *dinamikusan állítja elő* a weblap HTML-szövegét.

A dinamikus weblapok tartalma ezért nem állandó, hanem *a program által feldolgozott adatoktól függ*.

Szerveroldali programról beszélünk akkor, ha a dinamikus weblapot készítő *programkód* egy *távoli gépen működik*, és hozzánk már csak a kész weblap jut el.

Statikus és Dinamikus weblapok

Weblapok *statikusak* és *dinamikusak* lehetnek:

*Statikus weblap*ról beszélünk akkor, ha annak tartalmát a *fejlesztő előre meghatározza*, *leírja* és *tárolja.*

Az ilyen weblapok tartalma minden egyes *letöltéskor, megtekintéskor* ugyanaz.

Statikus és Dinamikus weblapok

A *dinamikus weblapokat*, vagy azok egy részét nemcsak a *fejlesztő,* hanem valamilyen *programkód* hozza létre.

A *program* általában *adatokat fogad*, és azok feldolgozása után *dinamikusan állítja elő* a weblap HTML-szövegét.

A *dinamikus weblapok* tartalma ezért nem állandó, hanem **a *program által feldolgozott adatoktól függ***.

Statikus és Dinamikus weblapok

Szerveroldali programról beszélünk akkor, ha a dinamikus weblapot készítő *programkód* egy *helyi/távoli* gépen működik, és hozzánk már csak a kész weblap jut el. *Szerver oldali*: PHP, ASP, Perl, CGI, Python.

Kliensoldali programról beszélünk akkor, ha a *kód* a *weblapba építve*, azzal *együtt töltődik le*, és *utasításait* a *böngésző hajtja végre*. Kliens oldali: *Javascript*

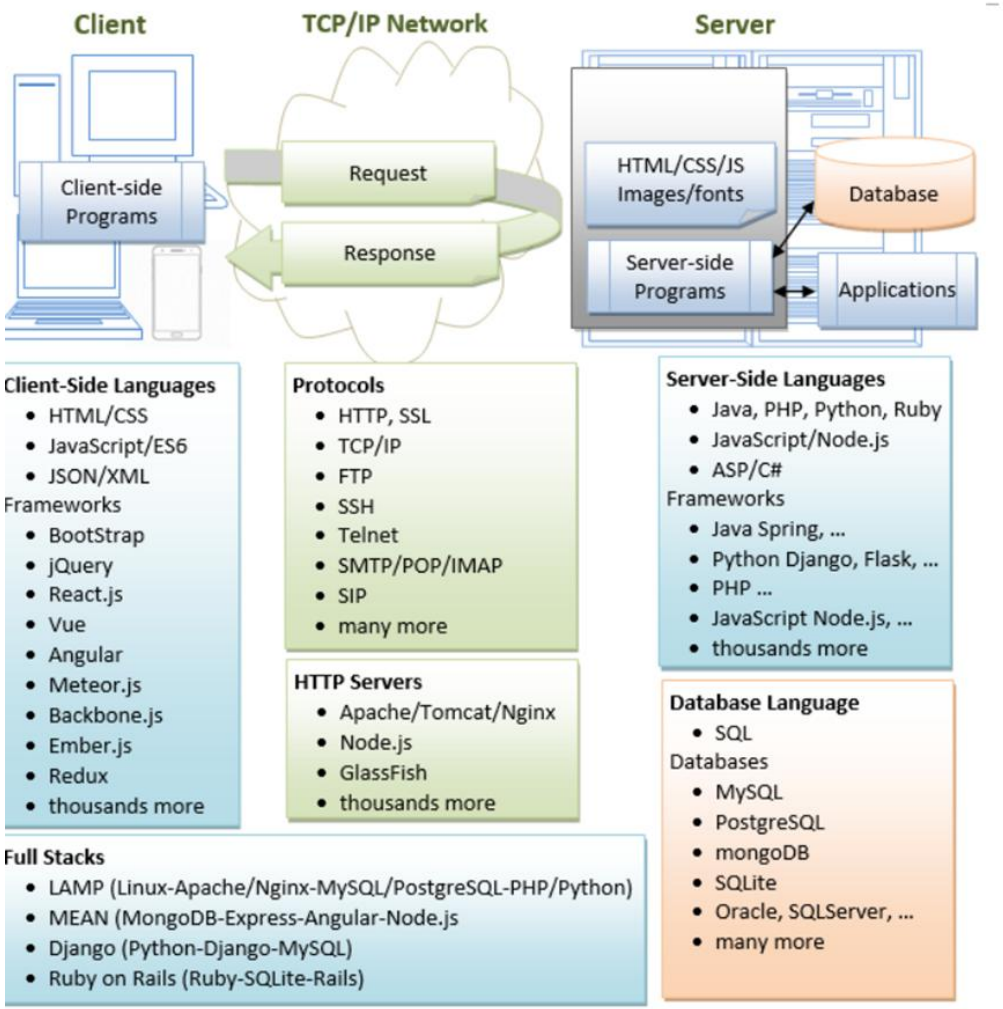
Full-Stack Web Developer

A *full-stack* webfejlesztő olyan személy, aki *mind kliens*, mind *szerver szoftvert* képes fejleszteni.

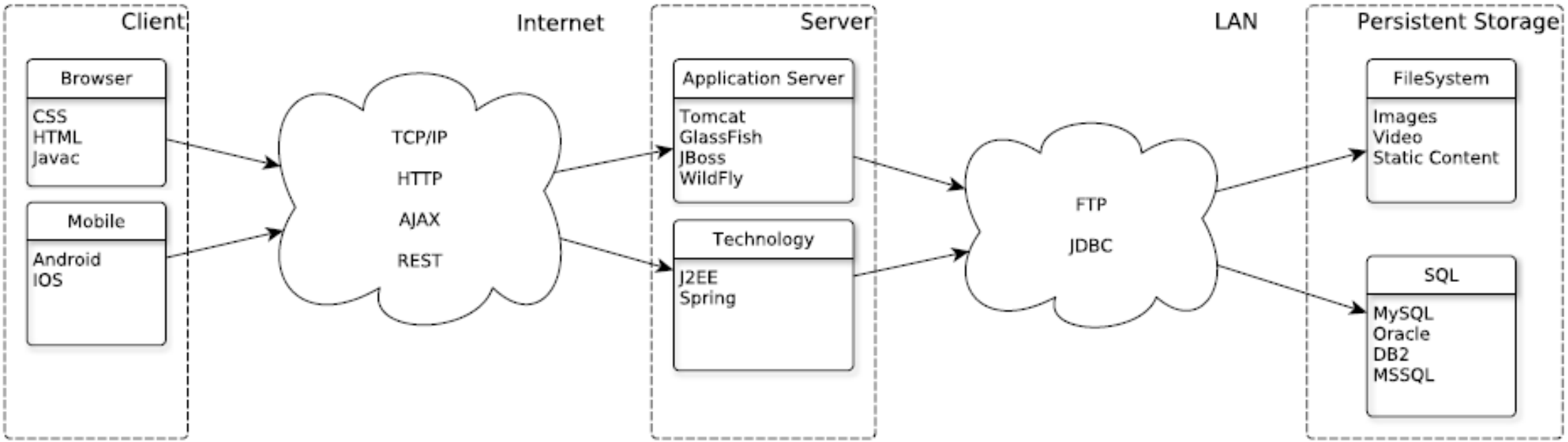
Ismernie kell:

- az ügyféloldali (front-end) programozást,
- a szerveroldali (back-end) programozást és az adatbázist.

Full-Stack Web Developer



Web-technológiák - Overview



Web Application are Distributed Systems

Web Application are Distributed Systems

- Front-end
- Back-end
- Database

Front-end

- Thin-, Thick Client

Back-end

- Could be Distributed

Database

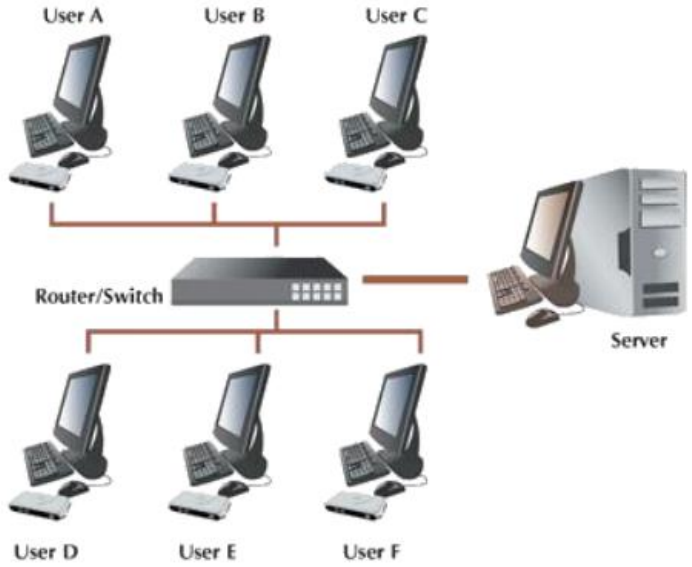
- Clusters, Replications

Vékony, vastag kliens

Vékonykliens:

- egy *minimális eszközökkel* rendelkező számítógép, mely
- a programok futtatásához at a távoli kiszolgálón, vagy kiszolgál

Előnye: alacsonyabb üzemelt



Vékony, vastag kliens

Vastag kliens:

Gyakorlatilag *egy szoftver,* mely a *kliens számítógépen* fut, az elvégzendő feladatokat a szerver és a kliens megosztva hajtja végre.

Önmaga hajt végre nagyobb adatmennyiségekkel feldolgozásokat.

Webes alkalmazások architektúrája - Front-end

Browser

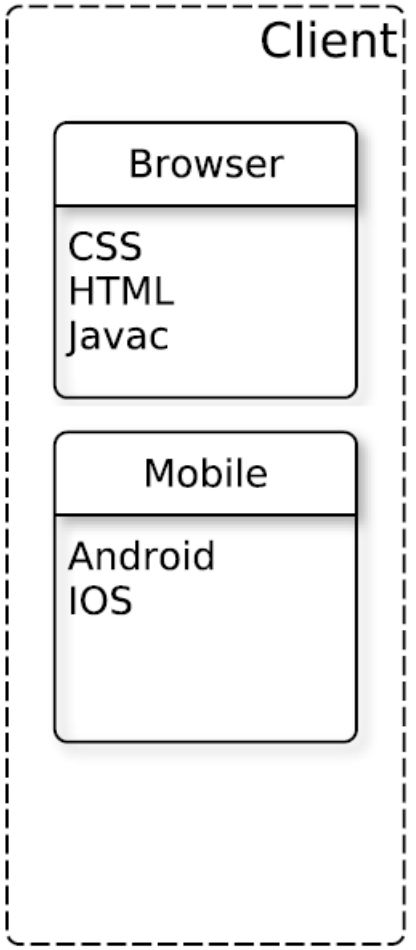
- HTML,
- CSS, Bootstrap
- JavaScript, AngularJS

Mobile

- Android
- iPhone

Desktop

- Thick clients



Front-end & Back-end Communication

Protocols

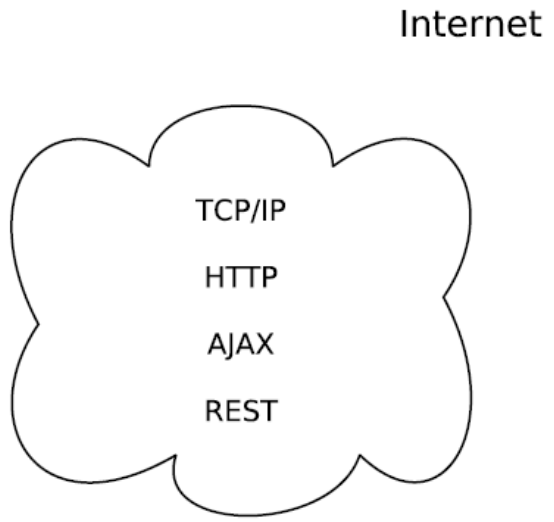
- TCP/IP
- HTTP, HTTPS

Techniques

- AJAX
- REST API (szoftverarchitektúra típus, elosztott kapcsolat, nagy, internet alapú rendszerek számára)

Data Format

- XML
- JSON (egy kis méretű, szöveg alapú szabvány)



Back-end

Script Languages

- PHP, Python, Ruby

Application Servers

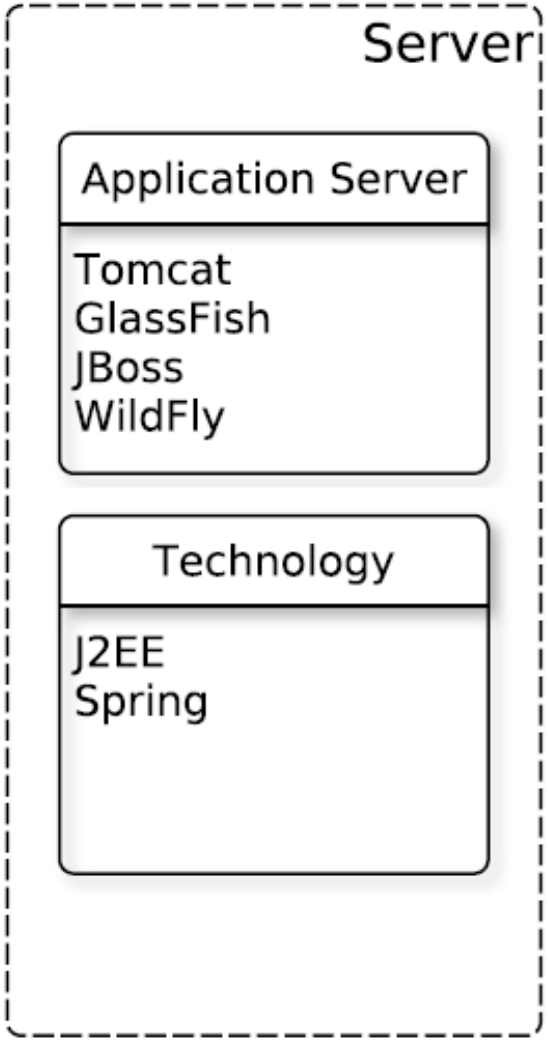
- Tomcat, GlassFish, JBoss, WildFly, Spring

Technology

- *Servlet Technology*
- *Spring Framework*

Deployment

- Single Instance
- *Integration*



Persistent Storage

File System

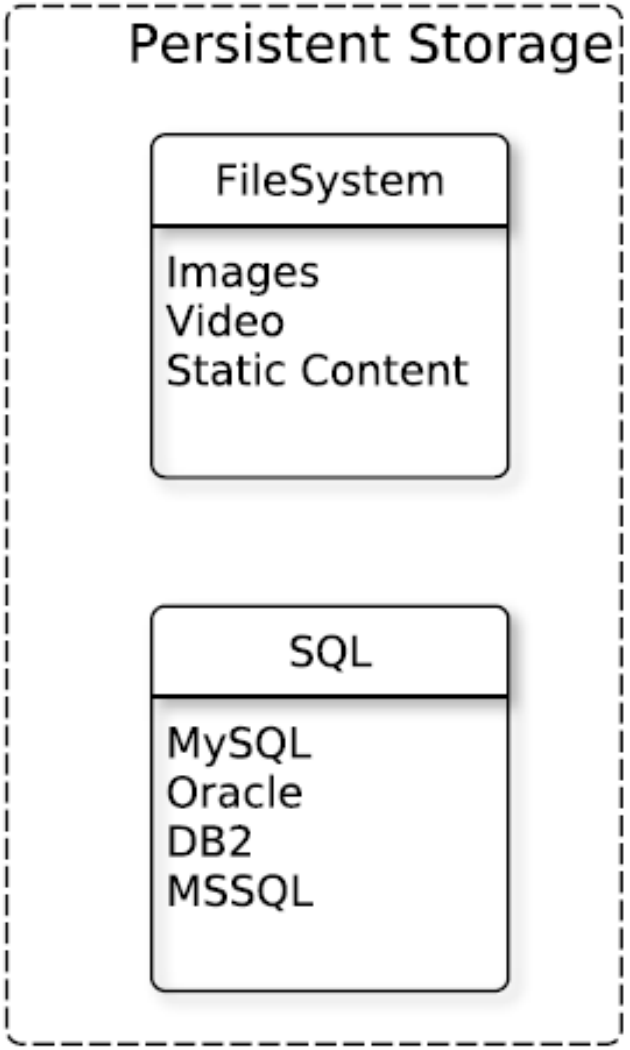
- Static Content
  - File, Image, Video, Audio

SQL

- MySQL, PostgreSQL
- Oracle, IBM DB2, SQLServer

NoSQL

- MongoDB
- Neo4j



Webalkalmazás technológiák - összefoglalás

Webszerver: olyan szerver alkalmazás, amely HTTP kéréseket fogad, értelmezi ezeket, majd a megfelelő válaszokat visszaküldi az ügyfél programoknak (böngészőknek).

Példa: Apache Web Server.

Alkalmazásszerver: olyan szerver, amely lehetőséget teremt üzleti komponensek futtatására is.

Pl.: Sun Application Server (Glassh), Bea WebLogic, Oracle Application Server stb.”[1][2][3][4][5][6] [7]

Felhasznált irodalom

[1] Szabó Bálint: Webprogramozás I. 2013.

[2] HTML Tutorial, https://www.w3schools.com/html/

[3] A CSS3 alapjai-példákkal bemutatva!

https://gremmedia.hu/css-alapjai-peldakkal-bemutatva#

[4] THE Moodle rendszer,

URL: https: //moodle.unithe.hu/course

Felhasznált irodalom

[5] GitHub cliens:

URL: https://git-scm.com/ or https://gitforwindows.org/

[6] HTML táblázat készítése

https://webiskola.hu/html-ismeretek/html-tablazat-keszites-html-table/

[7] CSS grid

https://szit.hu/doku.php?id=oktatas:web:css:css_grid